arXiv:2604.13323v1 [cs.RO] 14 Apr 2026

Vectorizing Projection in Manifold-Constrained Motion Planning for Real-Time Whole-Body Control

Shrutheesh R. Iyer, I-Chia Chang, Andrew Z. Liu, Yan Gu, and Zachary Kingston

***Abstract*— Many robot planning tasks require satisfaction of one or more constraints throughout the entire trajectory. For geometric constraints, manifold-constrained motion planning algorithms are capable of planning collision-free path between start and goal configurations on the constraint submanifolds specified by task. Current state-of-the-art methods can take tens of seconds to solve these tasks for complex systems such as humanoid robots, making real-world use impractical, especially in dynamic settings. Inspired by recent advances in hardware accelerated motion planning, we present a CPU SIMD-accelerated manifold-constrained motion planner that revisits projection-based constraint satisfaction through the lens of parallelization. By transforming relevant components into parallelizable structures, we use SIMD parallelism to plan constraint satisfying solutions. Our approach achieves up to 100–1000x speed-ups over the state-of-the-art, making real-time constrained motion planning feasible for the first time. We demonstrate our planner on a real humanoid robot and show real-time whole-body quasi-static plan generation. Our work is available at https://commalab.org/papers/mcvamp/.**

I. Introduction

Constrained motion planning problems arise commonly in many robotic tasks. Consider the problem of transporting a cup of water; the robot must find a trajectory in which the mug remains upright throughout to prevent spillage. Or, consider a humanoid robot transporting a large object. The robot must maintain balance at all times and keep both arms attached to the box to preserve feasible motion. Many of these constraints can be represented as implicit functions, which give rise to a submanifold of valid configurations in the robot's configuration space. Manifold-constrained motion planning is thus the problem of finding feasible, collision-free trajectories that satisfy a constraint imposed by a desired task from a start configuration to a goal region.

Despite their prevalence, manifold-constrained problems remain difficult to solve in real-time. The primary bottleneck lies in the ability to find constraint satisfying configurations quickly, as they typically are not explicitly defined. This problem is also exacerbated by the dimensionality of the system, the number of constraints that must be simultaneously satisfied, and the number of obstacles in the scene. For example, consider the Digit robot (shown in Fig. 1), a 30 degree-of-freedom (DoF) humanoid with a 6-DoF floating base. In this quasi-static whole-body coordination problem, the robot must maintain balance, have both feet on the ground at all times, and preserve the closed-chain constraint to keep both arms holding the box during transport. Consequently, most constrained motion planning works take anywhere in the order of hundreds of milliseconds to tens of seconds to solve, which limits real-time planning and execution.

SRI, AL, and ZK are with Department of Computer Science, Purdue University, {iyer270, liu3447, zkingston}@purdue.edu. ICC and YG are with School of Mechanical Engineering, Purdue University, {chang970,yangu}@purdue.edu

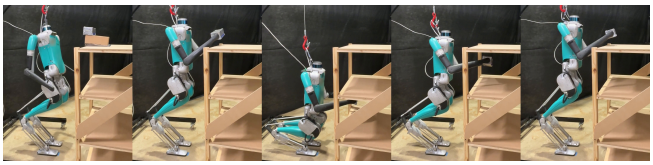


Fig. 1: Real-time quasi-static whole-body planning with our vectorized planning approach on the Digit humanoid. In this task, the robot must transport a box between the three levels of the shelf, which requires whole body coordination, obstacle avoidance, and constraint satisfaction. Our planner finds feasible motion from start to goal in this scenario in less than 12 milliseconds on average, practical for real-time planning.

Recent advances in accelerating motion planning have made real-time motion planning feasible; many approaches have used CPU and GPU acceleration to bring planning time to the millisecond time scale [1–3]). Most relevant to our approach is the Vector-Accelerated Motion Planning (VAMP) library [4], which exploits CPU "Single Instruction, Multiple Data" (SIMD) parallelism and data parallelism to achieve multiple orders-of-magnitude speed-ups by parallelizing forward kinematics and collision checking in motion validation, bringing planning times for unconstrained geometric planning problems into the realm of microseconds. However, while promising, these approaches either do not consider task constraints nor scale to very high-DoF systems.

To this end, we revisit the problem of samping-based manifold-constrained motion planning through the lens of vector acceleration. The primary bottleneck—beyond collision checking—is the manifold-constrained extension step, i.e., tracing a geodesic between configurations on a manifold. We tackle this with *fine-grained parallelization* of the projection of configurations in a geodesic onto the manifold. Altogether, we present the **Manifold-Constrained Vector Accelerated Motion Planner (McVAMP)** which is capable of planning on the order of microseconds to milliseconds for a wide range of manifold-constrained systems including whole-body manipulation. We believe this is a categorical shift in capabilities of planning for constrained high-DoF systems, as planning in real-time enables reactive global planning which could be used both for high-level task planning as well as low-level control.

Thus, our contribution is as follows

1) We present a single-core CPU-only SIMD-accelerated sampling-based manifold-constrained motion planner

that is capable of planning in the order of microseconds to milliseconds.
2) At the heart of our planner is a SIMD-parallel manifold-constrained extension step, which projects multiple configurations in parallel to satisfy constraints, and then evaluates for collision building upon VAMP's existing primitives.
3) We demonstrate the ability of our planner to compose multiple constraints and develop a quasi-static whole-body motion planner for a humanoid capable of producing stable, feasible motion plans that a robot with a classical controller can execute without modification.

We evaluate our planner on a number of challenging problems in simulation and reality with a 7-DoF arm, a 14-DoF bimanual system, and a 28-DoF humanoid. Our approach outperforms the existing state-of-the-art baselines by orders of magnitude in terms of speed, while obtaining a better success rate and similar path quality.

II. Preliminaries

The goal of constrained motion planning is to compute a collision-free trajectory between start and goal configurations while adhering to constraints imposed by the robot or task specifications. Formally, let $\mathcal{C} \subset \mathbb{R}^d$ be the robot configuration space, where d typically represents the number of actuated joints and $q \in \mathcal{C}$ is a robot configuration. The Euclidean space $\mathbb{R}^d$ is referred to as the ambient space. The obstacle-free configuration space is represented by $\mathcal{C}_{\text{free}} \subset \mathcal{C}$. Let $\mathcal{F}_i(q) : \mathcal{C} \rightarrow \mathcal{R}^k$ denote the ith constraint imposed on the robot, which is a function of the robot's configuration and k represents the dimensionality of the constraint. A configuration q satisfies the constraint if $\mathcal{F}_i(q) = \mathbf{0}$. In practice, an acceptable numerical tolerance is imposed, $\mathcal{F}_i(q) \leq \epsilon$. Each constraint reduces the dimensionality of $\mathcal{C}$, and defines a $(d-k)$-dimensional submanifold $\mathcal{M}_i$ embedded within $\mathcal{C}$. We also consider the Jacobian $J_i(q)$ of the constraint function $\mathcal{F}_i(q)$, which is the matrix of first-order partial derivatives with respect to q. While we refer to each constraint function individually, we also consider their composition, $\mathcal{F}(q)$, and corresponding composite manifold $\mathcal{M}$. Thus, the goal of constrained motion planning is to find a collision-free trajectory $\sigma(t) : [0,1] \rightarrow \mathcal{M}_{\text{free}}$ from q_a to q_b such that it is both collision free and lies on the manifold, given that $q_a, q_b \in \mathcal{M}_{\text{free}} (= \mathcal{M} \cap \mathcal{C}_{free})$.

Inspired by Berenson et al. [5], we model end-effector pose constraints as Task Space Region (TSR) constraints. Let $\mathcal{T} \in SE(3)$ denote a pose transform, where $\mathcal{T}_b^a$ transforms from frame a to b, and is composed of a translational and rotational component $[t_b^a, R_b^a]$. A TSR is composed of two transforms $\mathcal{T}_w^0$, $\mathcal{T}_e^w$, and bounds $\mathcal{B}^w$

- $\mathcal{T}_w^0$ transforms global frame to the TSR frame w (i.e., the object frame),
- $\mathcal{T}_e^w$ transforms object frame to the end-effector frame,
- $\mathcal{B}^w$ represents the bounds for the rotational and translation errors. This specifies which components of the $SE(3)$ pose of the end-effector are constrained.

For example, for a grasped mug to be held upright, $\mathcal{T}_w^0$ is the mug pose, $\mathcal{T}_e^w$ represents the grasp pose, and $\mathcal{B}^w$ specifies that the mug is free to translate and rotate about the z-axis, but is tightly constrained along the x- and y-axes.

Using this formulation, for a given configuration q, the constraint function $\mathcal{F}(q)$ is expressed as the distance to the constraint manifold, given by:

$$T_{s'}^w = (T_w^0)^{-1} T_s^0 (T_e^w)^{-1}, \qquad d^w = \begin{bmatrix} t_{s'}^w \\ \log(R_{s'}^w)^\vee \end{bmatrix} - B^w \tag{1}$$

where $T_s^0 = FK(q)$ is the end-effector pose of q obtained by forward kinematics, $T_{s'}^w$ denotes the pose error in $SE(3)$, $d^w \in R^6$, and $\log(R_{s'}^w)^\vee$ represents the logarithmic map of the $SO(3)$ rotational element of the standard Lie algebra.

III. Related Work

Manifold-constrained motion planning has been studied extensively. Sampling-based planners compute plans by sampling from $\mathcal{C}_{\text{free}}$ without explicitly computing the space, including multi-query planners like PRM [6] and single-query planners like RRT [7]. These methods extend to constrained planning by sampling constraint-satisfying configurations, which fall into two major categories of approach. Projection-based methods use the constraint Jacobian to find satisfying configurations, e.g., CBiRRT2 [5] uses Jacobian pseudoinverse projection within bidirectional RRT. Continuation methods construct approximations of the manifold using the Jacobian to construct tangent spaces, such as in AtlasRRT [8]. We refer the reader to Kingston et al. [9] for a full survey. Despite progress, efficient manifold sampling remains a bottleneck for real-time planning.

Learning-based methods show promise for acceleration, e.g. CompNetX [10] and C-NTFields [11] learn to sample on task-specified manifolds or learn the manifold structure. However, these require large amounts of training data and cannot easily adapt to new constraints. Another approach samples directly from manifolds using inverse kinematics (IK) [12], though IK computation often bottlenecks performance. Cohn et al. [13] reformulate bimanual constraints using an analytic IK formulation which reparameterizes the constraint, enabling planning through the Graph of Convex Sets (GCS) method [14]. However, this restricts solutions to a single C-bundle and requires extensive pre-computation.

A. Parallelized Motion Planning

There are many approaches to parallelizing motion planning; multiple aspects of planning are trivially parallelizable such as running many searches [15] or many iterations in parallel [16, 17]. GPU approaches [17] merge multiple parallelism levels. The cuRobo [1] planner uses GPU-parallel multi-seed trajectory optimization, imposing soft penalties for constraint adherence as is typical [18, 19] However, soft formulations do not guarantee strict manifold adherence.

Our approach builds off of VAMP [4], which performs vectorized collision checking through CPU SIMD, showing tremendous speedup. Another recent work, cPRRTC [20],

showed that parallelizing projection can accelerate constrained motion planning. Our approach demonstrates that VAMP's insight of batching validation across an edge holds also for manifold projection, providing order-of-magnitude speedups over baseline approaches.

B. Whole Body Planning

Whole-body humanoid planning must satisfy multiple constraints (balance, collision avoidance, task poses) for high-DoF systems (>25 DoF), making these spaces highly non-convex. For multiple constraint satisfaction, Sentis and Khatib [21] use hierarchical null-space projection, which does not handle conflicting constraints well. Kaiser et al. [22] introduce the Constellation technique for conflicting constraints, which Agrawal et al. [23] build upon by proposing constrained nonlinear Kaczmarz for large constraint sets. We use cyclic projection in our approach to handle many constraints simultaneously.

Specific to humanoids, planners such as CBiRRT [5] have tackled the problem, but are not fast enough for real-time scenarios. Lembono et al. [24] speed up planning by learning feasible configurations of the system, but require large amounts of training data. IK-based reactive control such as [25] are fast but often get stuck in local minima.

IV. Methodology

In manifold-constrained sampling-based motion planning, local connections between configurations should satisfy constraints. Typically, these are approximations of a geodesic on the constraint submanifold, and are computationally expensive to generate. Our primary contribution is vectorizing the generation of constraint satisfying motion by parallelizing the projection step that maps configurations onto the constraint manifold. The key insight is that multiple configurations along an extension can be projected and checked in parallel using SIMD operations, dramatically reducing the overall cost of constraint satisfaction. We first present the complete manifold-constrained RRT-Connect algorithm, then detail the vectorized projection mechanism, and finally discuss the choices that went into the design of our planner.

Algorithm 1 Vectorized Manifold-Constrained RRT-Connect

Require: $q_a, q_b, \mathcal{F}$
1: $T_\text{a} \leftarrow \text{Tree}(q_a)$, $T_\text{b} \leftarrow \text{Tree}(q_b)$
2: **while** iter $<$ MaxIterations **do**
3: $q_\text{rand} \leftarrow \text{RandomConfig}()$
4: $q^a_\text{near} \leftarrow \text{Nearest}(T_\text{a}, q_\text{rand})$
5: $q^a_\text{proj} \leftarrow$ ParallelConstrainedExtend$(T_\text{a}, q^a_\text{near}, q_\text{rand})$
6: **while** $q^a_\text{proj} \neq$ NULL **do**
7: $q^b_\text{near} \leftarrow \text{Nearest}(T_\text{b}, q^a_\text{proj})$
8: $q^a_\text{proj} \leftarrow$ ParallelConstrainedExtend$(T_\text{a}, q^a_\text{proj}, q^b_\text{near})$
9: **if** $q^a_\text{proj} = q^b_\text{near}$ **then**
10: P $\leftarrow \text{ExtractPath}(T_\text{a}, T_\text{b})$
11: **return** P
12: $\text{Swap}(T_\text{a}, T_\text{b})$
13: iter $\leftarrow$ iter $+ 1$
14: **return** NULL

Algorithm 2 ParallelConstrainedExtend

Require: $T, q_s, q_\text{target}, \mathcal{F}, r, \sigma, n =$ SIMD width (e.g., 8 for Intel AVX)
1: **procedure** ConstrainedExtend
2: $\text{dist} \leftarrow \min(\|q_\text{target} - q_s\|, r)$
3: $v_\text{extend} \leftarrow \frac{q_\text{target} - q_s}{\|q_\text{target} - q_s\|}$
4: $q_\text{steer} \leftarrow q_s + \text{dist} \cdot v_\text{extend}$
5: $q^{(i)}_\text{particles} \leftarrow q_\text{steer} + \epsilon_i v_\text{extend},\ \epsilon_i \sim \mathcal{N}(0, \sigma^2),\ i = 1, \ldots, n$
6: $q_\text{proj} \leftarrow \text{ParallelProject}(q_\text{particles})$
7: **if** $q_\text{proj} =$ NULL **then**
8: **return** NULL
9: **if** $\neg\text{CollisionFree}(q_\text{proj}) \vee \|q_\text{proj} - q_s\| > 2 \cdot \text{dist}$ **then**
10: **return** NULL
11: $q_\text{interp} \leftarrow n$ interpolated points between q_s and q_proj
12: $q_\text{int_proj} \leftarrow \text{ParallelProject}(q_\text{interp})$
13: **if** $\exists i : \|q^{(i)}_\text{int_proj} - q^{(i+1)}_\text{int_proj}\| > \frac{r}{n}$ **then**
14: **return** NULL ▷ *Projected points too far apart*
15: ▷ *Recursively interpolate and project until resolution δ is met* ◁
16: $T.\text{addVertex}(q_\text{proj})$
17: $T.\text{addEdge}(q_s, q_\text{proj})$
18: **return** q_proj
19: **procedure** ParallelProject
20: **while true do**
21: $d \leftarrow \mathcal{F}(q_\text{particles})$
22: **if** $\forall i : \|d^{(i)}\| < \epsilon$ **then**
23: **return** $q_\text{particles}$ ▷ *All particles converged*
24: $\text{step} \leftarrow \text{getDescentStep}(q_\text{particles})$
25: **if** $\exists i : \|\text{step}^{(i)}\| > \text{MaxDistance}$ **then**
26: **return** NULL ▷ *Projection diverging*
27: $q_\text{particles} \leftarrow q_\text{particles} + \text{step}$
28: **procedure** getDescentStep
29: $q_\text{init} \leftarrow q_\text{particles}$
30: **for** $\mathcal{F}_i \in \{\mathcal{F}_1, \ldots, \mathcal{F}_k\}$ **do**
31: $d \leftarrow \mathcal{F}_i(q_\text{particles})$
32: $J_i \leftarrow$Jacobian of $\mathcal{F}_i(q_\text{particles})$
33: $\Delta q \leftarrow \alpha J_i^T (J_i J_i^T + \lambda I)^{-1} d \quad (\text{or} \quad \alpha (J_i^T J_i + \lambda I)^{-1} J_i^T d)$
34: $q_\text{particles} \leftarrow q_\text{particles} - \Delta q$
35: **return** $q_\text{particles} - q_\text{init}$

A. Vectorized Projection-Based Sampling-Based Planning

Our parallel vectorized manifold-constrained RRT-Connect algorithm is presented in Alg. 1. This is a variant of the RRT-Connect algorithm [7] where all nodes and edges lie on the manifold $\mathcal{M}_\text{free}$ up to some discretization resolution δ. In practice, we also use the dynamic-domain [26] and balancing heuristics [27], but have elided these from the pseudocode for clarity.

First, a q_{rand} is sampled from the ambient space, and its nearest neighbor q_{near} is found (Alg 1, Line 4). The ParallelConstrainedExtend method attempts to connect q_{near} to q_{rand} (Alg 1, Line 5). If the extension is successful, the planner repeatedly attempts to grow the projected configuration toward the goal tree using the same ParallelConstrainedExtend method with a fixed extension step size.

The key contribution of our work is the vectorized ParallelConstrainedExtend method (Alg. 2). Following the intuition and empirical results from VAMP [4] we focus on the constrained motion validation step, and parallelize the projection and validation of interpolated configurations. However, this is not trivial since each point along the extension vector (i) could be at different distances from the manifold, and (ii) have to be projected such that there is a continuous path on the manifold between the projected points. To deal

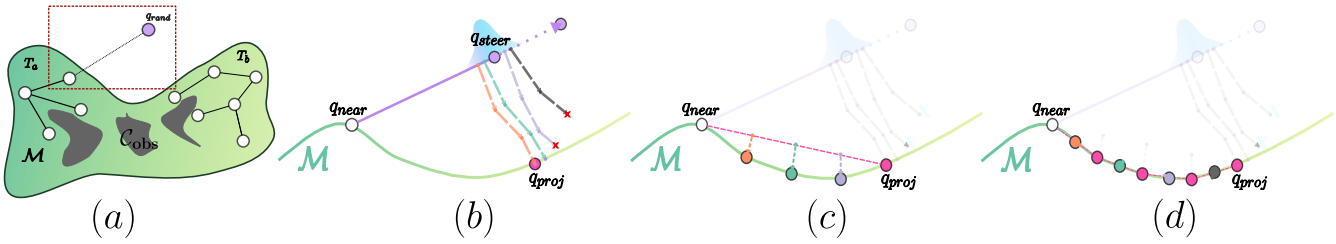


Fig. 2: Methodology. (a) RRT-Extend Step: A random configuration in ambient space is sampled. (b) q_{steer} is computed at a fixed distance from q_{near} and samples around q_{steer} are projected onto the manifold until any one succeeds. (c) Interpolated samples along the vector connecting the start and the initial projected point are projected in parallel. (d) Configurations are interpolated between the projected particles and are projected and validated recursively until desired resolution is achieved. $n = 4$ *here for illustrative purposes, each represented by a different color.*

with these issues, we present a two step vectorized approach. Throughout the rest of the discussion, n denotes the number of parallelized operations.

First, q_{steer} (Alg 2, Line 4) is computed at a fixed distance away from q_{near} along the vector q_{nr}. Then we sample n points around q_{steer} (Alg 2, Line 5) on direction vector, and attempt to project all points onto the manifold. We exit if any of them succeed and record the particle that succeeds the earliest, as q_{proj} (Alg 2, Line 6). Then, n points are linearly interpolated between q_{near} and q_{proj} and projected onto the manifold (Alg 2, Line 11) in parallel. The next batch of n points are recursively interpolated and projected between adjacent projected points until the desired motion validation resolution is achieved. Collision checking is performed in parallel after projection, and we exit early if any projected point is in collision.

Sine the number of points to check along the motion vector is typically small (on the order of 8–64), this is perfectly suited to SIMD parallelization, which provides a high throughput without significant overhead, and allows flexible interleaving of sequential and parallel code, something that an iterative optimization method benefits from. To support data parallelism necessary for SIMD instructions, all configurations, function evaluations, and Jacobians are stored in a structure-of-arrays layout.

The critical component of the parallel extend method is parallel projection of multiple points onto the manifold. To be able to perform this, a few pieces are needed.

Tracing Compiler for Constraints In a manner similar to VAMP, we generate SIMD parallel code for evaluating constraint functions using a tracing compiler to *trace* the low-level operations needed to compute the `distanceToConstraint` (Alg 2, Line 21). We use build upon the existing tracing compiler used by VAMP which uses Pinocchio [28] and CppAD [29] to generate efficient, branch-free, loop-unrolled code for robot kinematics, which is amenable to SIMD operations. By using automatic differentiation during the tracing process, we efficiently compute the corresponding Jacobian matrices of the constraint functions. To enable gradient-based optimization, we implement a differentiable version of $\log(R_{s'}^{w})^{\vee}$ by using a first order Taylor Series approximation of the `sinc` function at singularities.

Vectorized Levenberg–Marquardt: In addition to tracing the constraint functions and Jacobians, we also subsequently trace one step of the Levenberg-Marquardt (LM) algorithm to be able to more efficiently perform the gradient descent in projection. We exploit the known dimensionality of the robot and the constraint functions by using a second-order LM algorithm. To achieve this, we trace compile the Jacobian matrix pseudoinversion for each constraint. Given the semipositive definite (SPD) formulation of the LM step, we implement a custom Cholesky decomposition method [30], which also follows the SIMD principle of branchless control flow, and the fixed dimensionality allows us to perform loop-unrolled Cholesky solve expressions that can be parallelized. This subroutine is compiled for each constraint and allows us to perform one LM step in parallel for multiple particles. We provide both the inner and outer matrix pseudoinversions, the former is bounded by the dimension of the manifold function while the latter is bounded by the dimension of the robot.

B. Design Choices

Vectorized projection provides a twofold benefit. First, we can quickly project and evaluate multiple configurations in parallel, which provides naively an n-times speed-up compared to the sequential counterpart. More importantly, the parallel step also allows us to invalidate infeasible configurations very quickly, reducing wasted computations, which leads to the empirical 100-times speed-up we see in our experiments. By checking future configurations further along the validate step, we can cheaply invalidate the entire motion if any particle fails to project or is in collision. This follows the ethos of sampling-based planning by quickly invalidating infeasible motion to focus search elsewhere.

Early Exits The independent parallel nature of projection can cause inconsistencies where the interpolated points project to different parts of the manifold. Validating the connection between these points requires further interpolation, which may become unbounded. We avoid this altogether by exiting projection early if any particle has a large descent step, which likely indicates that the particles may be quite far from each other. Alg 2, Line 9 perform this early exit at different levels and invalidate the entire motion validation if any particle violates a projection distance threshold.

Two-Stage Projection While the two-stage projection may seem unnecessary at first (as prior projection-based planners simply interpolate towards q_{steer}), for hard constraints this reduces the likelihood of interpolated points projecting far away from each other during descent, as they are already quite close to the manifold. We provide an ablation study of

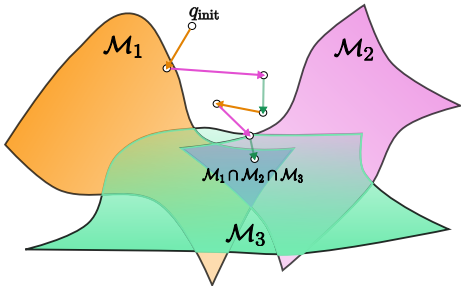


Fig. 3: Cyclic Projection to intersection of manifolds. At each iteration, we take one step towards each manifold in a pre-determined order, and the algorithm stops when we reach their intersections

this in Sec. V-B. The two-step projection also behaves as an adaptive range parameter for extension step: by sampling points around q_{steer}, depending on the difficulty of the constraint, a point that is either closer or farther away from the manifold could be projected the earliest, and overall provides higher chance of successful extension.

Consequently, our tree growth approach is built with the following principles: (i) stay close to the manifold while extending to limit the amount of computationally expensive projection iterations, and (ii) employ particle-based optimization to improve the success of manifold sampling, as even when the initial seeds are proximal in the ambient space the local gradient landscape can lead to divergent behavior.

C. Composition of Constraints

Some problems require satisfying multiple constraints, e.g., a humanoid carrying an object, as we explore in Sec. V-C. Thus, we need to be able to support multiple constraints, which translates to projecting onto the intersection of the manifolds, as seen in Fig. 3. Naively, these constraints can be combined into one large system of equations, but this scales poorly, as noted by [23]. We use a cyclic projection technique [31], described in Alg 2, Line 30. During each iteration of optimization, we compute the descent direction of one constraint and takes a step in that direction. Constraints are cycled through in a predetermined order, and this step is repeated until we converge onto the intersection of all constraint manifolds. Although it is a simple approach, it empirically works well, since its simplicity allows parallelization at no additional cost.

V. Experimental Evaluation

We evaluate our planner on a wide suite of problems with different types and complexities of constraints. We test it out on 3 different robots: the Franka Emika Panda Arm (7-DoF), a bimanual Kuka IIWA system (14-DoF), and the Digit Robot (36-Dof, including a 6-DoF floating base). We primarily evaluate our planner against the projection-based manifold-constrained RRT-Connect implementation from OMPL [32, 33] and CuRobo [1] for the Panda arm. We perform hyperparameter sweeps on all planners on each problem and choose the best performing configuration. For fair comparison, OMPL uses the collision checking of VAMP; constraint evaluation is done using Pinocchio [28]. For the Bimanual KUKA IIWA arm, we compare against the approach of Cohn et al. [13]. All algorithms were tested on an 5.4Ghz Intel i7-13700K CPU with 64GB of RAM and an NVIDIA RTX 4090 GPU with 24GB VRAM (for CuRobo).

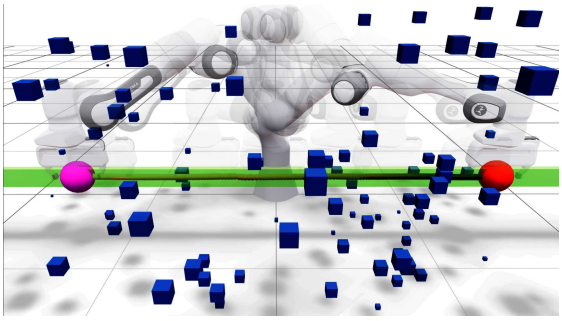


Fig. 4: Example of a line problem generated with start goal pairs (spheres) and the solution. Random cuboid obstacles are generated around the robot which have to be avoided.

A. End-Effector Constraint for a Single Arm

We first evaluate our constrained planner on the 7-DoF Franka Emika Panda arm. We test on 4 constraints: keeping the end-effector on a plane (i) without an orientation constraint and (ii) with a fixed orientation, and keeping the end-effector on a line (iii) without and (iv) with an orientation constraint. An example problem can be seen in Fig. 4. To generate the problems, we sample from a set of pre-defined lines and planes as constraints and sample valid start and goal configurations on the constraint. For each start-goal pair we incrementally add obstacles in the task space. Finally, we run our planner with many different hyperparameters and retain the problem even if any succeed. Note that CuRobo's constrained planner does not allow providing joint space configurations goals; we provide the end-effector pose of the goal configuration. We set a timeout of 10 seconds for OMPL after which the planner reports failure.

Fig. 5 shows the results for each class of constraint problem over a range of obstacle densities. In general, across all benchmarks we obtain superior performance in terms of planning time, with 100% success even as the difficulty increases. Most plans are under 1ms, which is anywhere from 100–2000× faster than the OMPL and CuRobo baselines.

Next, we evaluate the effectiveness of our planner to navigate complex environments while respecting constraints using a maze (shown in Fig. 6). We initialize 100 random start goal pairs at different points on the maze a minimum distance apart. We attach a marker to the end effector of the robot, whose tip is constrained to the bottom plane of the maze. In addition, we also impose an orientation constraint, such that only the yaw is free (i.e., the marker is free to rotate about its axis). We benchmark our algorithm against OMPL. As an ablation study, we replace the Pinocchio-based projection of OMPL with our compiled projection. For OMPL, we set the timeout to 100 seconds.

From the results in Table I, our planner shows a remarkable improvement on planning times and success rates, achieving over a 1000× speed-up in some cases, opening the door to real-time planning for hard manifold constraint

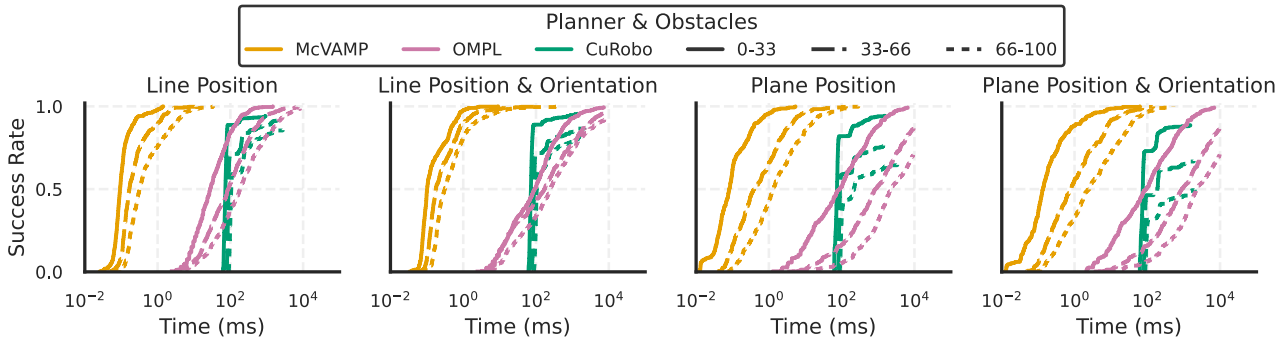
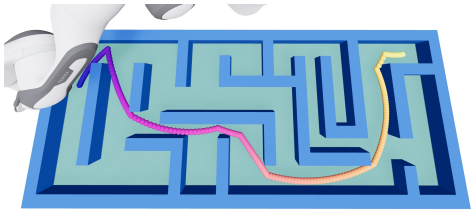
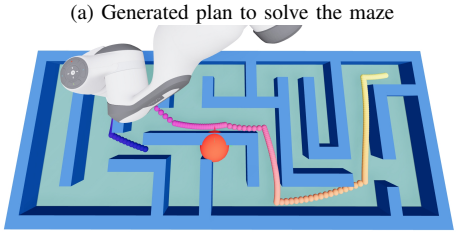


Fig. 5: CDF plot benchmarking planner performance for the line and plane constrained problems, binned by number of environment obstacles. *1. LP - Line with Position Constraint only, 2. LPO - Line with Position and Orientation Constraint, 3. PP - Plane with Position Constraint, 4. PPO - Plane with Position and Orientation Constraint* For most problems, McVAMP is **1000** times faster than the baselines achieving a 100% success rates, while the other planners suffer in the success rate as the number of obstacles increases.



(a) Generated plan to solve the maze



(b) Real-time replanning for dynamic environments

Fig. 6: Solving a Maze. Here the tip of the marker is constrained to the floor of the maze. Consequently, the robot has to solve the maze to find a plan from start to goal. Planning is done at 20Hz. (b) when a dynamic obstacle blocks the path (red sphere here), the planner can compute a new path due to the high motion validation and planning throughput

problems. We illustrate this in Fig. 6b where the planner is able to avoid a dynamic obstacle that blocks its current path.

Method	Mean	Q1	Median	Q3	95%	Succ.
OMPL	25.10	7.33	17.323	29.20	85.82	72%
OMPL (comp.)	13.25	1.00	3.29	16.80	52.07	76%
McVAMP	**0.032**	**0.002**	**0.006**	**0.014**	**0.058**	**82%**

TABLE I: Results for maze Solving. All times are reported in **seconds**. OMPL (comp.) refers to the OMPL planner with our compiled projection.

B. Bimanual Arm Constraint

To test the scalability of the planner to higher dimensions, we evaluate it on a box transport problem with a bimanual system. In this task, the 14-Dof arms are constrained such that the relative transform between the left and the right arm must be fixed throughout the motion, i.e., holding the box. To enforce this constraint, we formulate it as a TSR constraint:

$$T_{err} = T_{ref}^{-1}(T_{left}^{-1}T_{right}), \quad d_{err} = \begin{bmatrix} t_{err} \\ \log(R_{err})^{\vee} \end{bmatrix} \tag{2}$$

Following Cohn et al. [13], we evaluate our planner on the same problem of moving the arms between shelves while maintaining the relative pose and avoiding collisions. For baselines, we use compare against both their IK-BiRRT planner as well as the IK-GCS approach; these both use a pre-computed analytic representation of the manifold constraint.

As this problem is more complex and higher-dimensional, we evaluate an ablation our two stage projection approach to prove its value. We test a single-stage projection, where points are interpolated between q_{near} and q_{steer} (McVAMP 1-step). Table II shows the results of the planners.

Our approach provides 30–60× speed-up for solving the task compared to IK-BiRRT, and more than a 10× speed-up compared to IK-GCS. This result is interesting as IK-GCS requires pre-processing (about 70 seconds) to compute the graph of convex sets, which we do not count for benchmarking. However, IK-GCS produces shorter paths, owing to their optimization-based formulation. Our 2-step projection also outperforms the 1-step parallel projection approach, which indicates that it is beneficial as the complexity of the constraints increase.

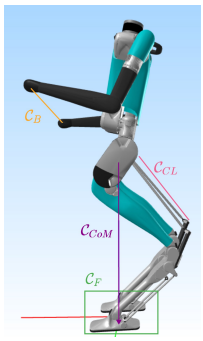
Method	T → M	M → B	B → T
IK-BiRRT	96.74 ± 48.48 (3.69 ± 0.63)	90.11 ± 21.70 (5.64 ± 0.96)	121.90 ± 53.61 (6.26 ± 1.09)
IK-GCS	63.47 ± 1.95 (**2.52 ± 0.0**)	44.18 ± 2.74 (**4.73 ± 0.0**)	77.87 ± 1.19 (**5.82 ± 0.0**)
McVAMP (1-step)	2.11 ± 1.16 (7.72 ± 1.50)	3.53 ± 2.44 (6.08 ± 1.51)	2.36 ± 1.63 (8.93 ± 1.33)
McVAMP	**1.65 ± 0.53** (7.62 ± 1.47)	**3.49 ± 1.21** (7.69 ± 2.97)	**1.37 ± 0.65** (8.28 ± 1.34)

T: Top, B: Bottom, M: Middle.

TABLE II: KUKA IIWA Bimanual Planning Results: Time in **milliseconds** (top) and Path Cost (bottom in parenthesis).

C. Whole Body Planning

In this scenario, we scale beyond beyond constraints on just the end-effector of a fixed-based manipulator and evaluate the ability to tackle composition of constraints. We implement our planner for the Digit robot to perform quasi-static whole body planning to transport a box. The



Constraint Equations

$\mathcal{C}_B$:	$T_{\text{err}} = T_{\text{ref}}^{-1}(T_L^{-1}T_R)$ $d_B = \begin{bmatrix} t_{\text{err}} \\ \log(R_{\text{err}})^\vee \end{bmatrix}$
$\mathcal{C}_{CL}$:	$d_{CL} = \begin{bmatrix} \|X^l_{\text{hip}} - X^l_{\text{tar}}\| \\ \|X^r_{\text{hip}} - X^r_{\text{tar}}\| \end{bmatrix}$
$\mathcal{C}_{\text{CoM}}$:	$x_{\text{near}} = \text{proj}(\mathbf{x}^{xy}_{\text{com}}, P)$ $d_{\text{COM}} = \mathbf{x}^{xy}_{\text{com}} - x_{\text{near}}$
$\mathcal{C}_F$:	$d^f = \begin{bmatrix} t^f_{\text{rel}} \\ \log((T^f_{\text{ref}})^{-1}T_f)^\vee \end{bmatrix}$ $d_F = \begin{bmatrix} d^l & d^r \end{bmatrix}^\intercal$

Fig. 7: The different constraints of the Digit robot. (i) $\mathcal{C}_B$, the bimanual fixed pose constraint, (ii) $\mathcal{C}_{CL}$, the closed linkage constraint, (iii) $\mathcal{C}_{COM}$, the center of mass constraint, and (iv) $\mathcal{C}_F$, the fixed feet placement constraint.

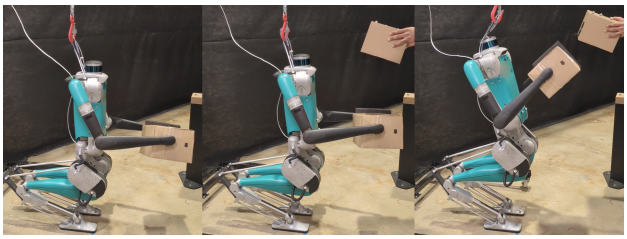
Digit robot has 30 joints and 6-DoF floating base. Of these 30 joints, 4 are high-stiffness springs, while another 6 are passive and constrained by closed-loop linkages. By assuming rigidity of the 4 spring joints and relying on the low-level tracking controller to handle the 4 closed-loop linkages associated with the ankles, we model it as a 28-Dof system, composed of 22 joints and a 6-Dof floating base, where 2 joints are passive and constrained by 2 closed-loop linkages. To generate a feasible motion plan for the Digit, the motion must respect the following four constraints (equations detailed in Fig. 7):

- **Stability Constraint:** To maintain balance, the xy-projection of the center of mass (CoM), $\mathbf{x}^{xy}_{com}$, must remain within the convex hull of the support polygon P. The error is to the nearest point in the hull.
- **Feet Constraint (TSR):** For fixed feet placement, we stack the 6-DoF pose errors for both feet ($f \in \{l, r\}$) into a 12-element vector.
- **Closed Link Constraint:** The closed loop linkages between the hip and the tarsus joint are enforced as fixed lengths between the two links.
- **Bimanual Constraint:** The relative pose between the hands is fixed, same as in Sec. V-B.

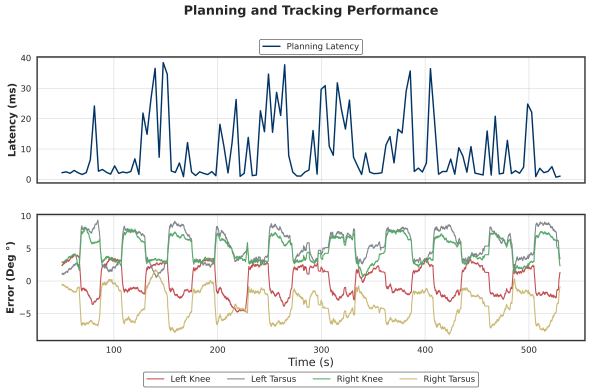
Satisfying the first three constraints is essential for the Digit, as they are required for stability and real-world operation. To control the robot to track the planned joint trajectory, a passivity-based controller [34] is used to generate feedforward motor torque commands that compensate for the ground reaction forces, and uses proportional-derivative feedback to correct tracking error.

We evaluate our planner in two real-world experiments. In additional to planning time, we also verify successful planning by reporting the tracking errors of relevant joints. A small tracking error indicates that the motion plan is more dynamically feasible, as the controller does not need to adjust motion to respect the stability, feet, and closed-link constraints. For these tasks, the planner was run on a 4.8GHz Intel Core Ultra 7 258V device with 32GB of RAM.

1) Box Transport: For this task, the robot is required to transport the box between three levels of the shelf, shown in Fig. 1. Each task is repeated 3 times; Table III shows the planning times for each task. For motion segment, we show



(a) Whole-body trajectory execution in dynamic environment. During the upward motion, an obstacle is detected, the robot plans a feasible trajectory around the obstacle.



(b) Tracking error and planning times for the repeated start-goal-start motions for the bimanual

Fig. 8: Experimental results for dynamic environment obstacle avoidance.

the distribution of the tracking error of the highest offending joints (the knee and the tarsus joints). Errors range less than 10 degrees on average, with a max error across all joints of 12 degrees at a single point for the left tarsus joint.

2) Dynamic Obstacle Avoidance: Here, the humanoid robot is tasked to repeatedly plan and execute a box transport task between fixed locations. A dynamic obstacle is introduced, and we test the ability of the robot to react and avoid the obstacles. Fig. 8a illustrates the motion of the robot, while Fig. 8b shows the tracking error and planning times.

Our planner is able to generate feasible trajectories in under 40ms, with the majority of planning times taking under 10ms, thereby enabling quick reactions to changing environments. All trajectories generated were feasible, and tracking errors remained under 8 degrees for the worst offending joints.

Motion phases	**S → T**	**T → B**	**B → M**	**M → T**
Tracking error (deg)				
Left Knee Joint	1.70 ± 1.57	0.64 ± 0.38	1.28 ± 0.34	1.00 ± 0.50
Left Tarsus Joint	4.37 ± 2.26	4.80 ± 1.82	8.63 ± 1.73	6.38 ± 1.31
Right Knee Joint	3.81 ± 1.78	2.57 ± 1.05	5.61 ± 0.71	4.51 ± 0.63
Right Tarsus Joint	3.51 ± 1.73	5.15 ± 1.93	7.74 ± 1.93	5.85 ± 1.89
Planning Time (ms)	4.93 ± 0.12	10.11 ± 1.65	13.42 ± 3.68	16.28 ± 2.11

Note: S: Standing Pose, T: Top, B: Bottom, M: Middle.

TABLE III: Planning to transport a box. The robot picks up the box from the top shelf, transports it to the bottom shelf, then the middle and finally the top one. We show the mean tracking error in the legs between the controller and the planner, and planning time in **milliseconds**. All numbers are reported as mean ± 1 standard deviation.

VI. Conclusion and Future Work

In this work, we present a vectorized manifold-constrained sampling-based motion planner that is capable of generating constraint-satisfying motion plans on the order of milliseconds, achieving over 500× speedup in challenging, cluttered, and high-dimensional problems. We also demonstrate the scalability of our approach by tackling a real-time, whole body control problem with a 28-DoF Digit humanoid, demonstrating reactive real-time constraint-satisfying planning in dynamic environments. We believe that this capability opens up many possibilities for real-world tasks, and potentially for tackling more complex planning problems such as whole-body task and motion planning for humanoid robots.

For future work, we are interested in investigating extensions to the manifold-constrained formulation that could be extended to kinodynamic planning, moving beyond the limitations of quasi-static planning. There are also improvements to the current planner that could be explored that are known in the literature, such as more sophisticated optimization techniques and integrating with continuation based methods. We also plan to investigate dynamic constraint and robot compilation, as currently our approach requires knowledge of constraints *a priori* to generate efficient SIMD kernels.

VII. Acknowledgement

We thank Thomas Cohn and Jiaming Hu for their help with baseline implementations and comparisons, and Jolie Lai for hardware experiments.

References

[1] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett, K. Van Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, et al. "cuRobo: Parallelized collision-free robot motion generation". In: *IEEE Int. Conf. Robot. Autom.* 2023, pp. 8112–8119.

[2] A. Du, E. Adabag, G. Bravo, and B. Plancher. "GATO: GPU-accelerated and batched trajectory optimization for scalable edge model predictive control". In: *arXiv preprint arXiv:2510.07625* (2025).

[3] L. Chen, S. R. Iyer, and Z. Kingston. "Differentiable Particle Optimization for Fast Sequential Manipulation". In: *arXiv preprint arXiv:2510.07674* (2025).

[4] W. Thomason, Z. Kingston, and L. E. Kavraki. "Motions in Microseconds via Vectorized Sampling-Based Planning". In: *IEEE Int. Conf. Robot. Autom.* 2024.

[5] D. Berenson, S. Srinivasa, and J. Kuffner. "Task space regions: A framework for pose-constrained manipulation planning". In: *Int. J. of Robotics Research* 30.12 (2011), pp. 1435–1460.

[6] L. Kavraki, P. Svestka, J.-C. Latombe, and M. Overmars. "Probabilistic roadmaps for path planning in high-dimensional configuration spaces". In: *IEEE Trans. Robot. Autom.* 12.4 (1996), pp. 566–580.

[7] J. Kuffner and S. LaValle. "RRT-connect: An efficient approach to single-query path planning". In: *IEEE Int. Conf. Robot. Autom.* Vol. 2. 2000, 995–1001 vol.2.

[8] L. Jaillet and J. M. Porta. "Path Planning Under Kinematic Constraints by Rapidly Exploring Manifolds". In: *IEEE Trans. Robot.* 29.1 (2013), pp. 105–117.

[9] Z. Kingston, M. Moll, and L. E. Kavraki. "Sampling-Based Methods for Motion Planning with Constraints". In: *Annual Review of Control, Robotics, and Autonomous Systems* 1.1 (2018), pp. 159–185.

[10] A. H. Qureshi, J. Dong, A. Choe, and M. C. Yip. "Neural manipulation planning on constraint manifolds". In: *IEEE Robot. Autom. Lett.* 5.4 (2020), pp. 6089–6096.

[11] R. Ni and A. H. Qureshi. "Physics-informed neural motion planning on constraint manifolds". In: *IEEE Int. Conf. Robot. Autom.* 2024, pp. 12179–12185.

[12] J. Wang, S. Liu, B. Zhang, and C. Yu. "Inverse kinematics-based motion planning for dual-arm robot with orientation constraints". In: *Int. J. of Advanced Robotic Systems* 16.2 (2019), p. 1729881419836858.

[13] T. Cohn, S. Shaw, M. Simchowitz, and R. Tedrake. "Constrained bimanual planning with analytic inverse kinematics". In: *IEEE Int. Conf. Robot. Autom.* 2024, pp. 6935–6942.

[14] T. Marcucci, M. Petersen, D. Von Wrangel, and R. Tedrake. "Motion planning around obstacles with convex optimization". In: *Science Robotics* 8.84 (2023).

[15] M. Otte and N. Correll. "C-FOREST: Parallel Shortest Path Planning With Superlinear Speedup". In: *IEEE Trans. Robot.* 29.3 (2013), pp. 798–806.

[16] J. Ichnowski and R. Alterovitz. "Parallel sampling-based motion planning with superlinear speedup". In: *IEEE/RSJ Int. Conf. Intell. Robot. Syst.* 2012, pp. 1206–1212.

[17] C. H. Huang, P. Jadhav, B. Plancher, and Z. Kingston. "pRRTC: GPU-Parallel RRT-Connect for Fast, Consistent, and Low-Cost Motion Planning". In: *arXiv preprint arXiv:2503.06757* (2025).

[18] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel. "Motion planning with sequential convex optimization and convex collision checking". In: *Int. J. of Robotics Research* 33.9 (2014), pp. 1251–1270.

[19] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa. "CHOMP: Gradient optimization techniques for efficient motion planning". In: *IEEE Int. Conf. Robot. Autom.* 2009, pp. 489–494.

[20] J. Hu, J. Wang, and H. Christensen. "cpRRTC: GPU-Parallel RRT-Connect for Constrained Motion Planning". In: *arXiv preprint arXiv:2505.06791* (2025).

[21] L. Sentis and O. Khatib. "Synthesis of whole-body behaviors through hierarchical control of behavioral primitives". In: *Int. J. of Humanoid Robotics* 2.04 (2005), pp. 505–518.

[22] P. Kaiser, D. Berenson, N. Vahrenkamp, T. Asfour, R. Dillmann, and S. Srinivasa. "Constellation-An algorithm for finding robot configurations that satisfy multiple constraints". In: *IEEE Int. Conf. Robot. Autom.* 2012, pp. 436–443.

[23] A. Agrawal, P. Mayer, Z. Kingston, and G. A. Hollinger. "Constrained nonlinear kaczmarz projection on intersections of manifolds for coordinated multi-robot mobile manipulation". In: *IEEE Int. Conf. Robot. Autom.* 2025, pp. 7726–7732.

[24] T. S. Lembono, E. Pignat, J. Jankowski, and S. Calinon. "Learning constrained distributions of robot configurations with generative adversarial network". In: *IEEE Robot. Autom. Lett.* 6.2 (2021), pp. 4233–4240.

[25] C. G. Atkeson, B. P. W. Babu, N. Banerjee, D. Berenson, C. P. Bove, X. Cui, M. DeDonato, R. Du, S. Feng, P. Franklin, et al. "No falls, no resets: Reliable humanoid behavior in the DARPA robotics challenge". In: *IEEE-RAS Int. Conf. on Humanoid Robots*. 2015, pp. 623–630.

[26] L. Jaillet, A. Yershova, S. M. La Valle, and T. Siméon. "Adaptive tuning of the sampling domain for dynamic-domain RRTs". In: *IEEE/RSJ Int. Conf. Intell. Robot. Syst.* 2005, pp. 2851–2856.

[27] J. J. Kuffner and S. M. LaValle. *An efficient approach to path planning using balanced bidirectional RRT search*. Tech. rep. Robotics Institute, Carnegie Mellon University, 2005.

[28] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiraux, O. Stasse, and N. Mansard. "The Pinocchio C++ library – A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives". In: *Int. Symp. on Syst. Integration*. 2019.

[29] B. Bell. *CppAD: a package for C++ algorithmic differentiation*. 2022.

[30] G. Dahlquist and Å. Björck. *Numerical methods*. Courier Corporation, 2003.

[31] P. Combettes. "The foundations of set theoretic estimation". In: *Proceedings of the IEEE* 81.2 (1993), pp. 182–208.

[32] I. A. Sucan, M. Moll, and L. E. Kavraki. "The Open Motion Planning Library". In: *IEEE Robotics & Automation Magazine* 19.4 (2012), pp. 72–82.

[33] Z. Kingston, M. Moll, and L. E. Kavraki. "Exploring implicit spaces for constrained sampling-based planning". In: *Int. J. of Robotics Research* 38.10-11 (2019), pp. 1151–1178.

[34] Y. Gong and J. W. Grizzle. "Zero dynamics, pendulum models, and angular momentum in feedback control of bipedal locomotion". In: *J. of Dynamic Systems, Measurement, and Control* 144.12 (2022), p. 121006.